ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

HỆ ĐIỀU HÀNH

CHƯƠNG 8: BỘ NHỚ ẢO

Trình bày các khái niệm tổng quan về bộ nhớ ảo, các kỹ thuật cài đặt bộ nhớ ảo như phân trang theo yêu cầu, phân đoạn theo yêu cầu và một số vấn đề khác trong bộ nhớ ảo như: frames và thrashing

PHAN ĐÌNH DUY

Các nội dung đã học

- Chương 1: Tổng quan về hệ điều hành
- Chương 2: Cấu trúc hệ điều hành
- Chương 3: Quản lý tiến trình
- Chương 4: Định thời CPU
- Chương 5: Đồng bộ hoá tiến trình
- Chương 6: Tắc nghẽn
- Chương 7: Quản lý bộ nhớ
- **Chương 8: Bộ nhớ ảo**
- Chương 9: Hệ điều hành Linux và Hệ điều hành Windows

MỤC TIÊU

1. Hiểu được các khái niệm tổng quan về bộ nhớ ảo
2. Hiểu và vận dụng các kỹ thuật cài đặt bộ nhớ ảo demand paging
3. Hiểu được một số vấn đề đề trong bộ nhớ ảo như: frames và thrashing

NỘI DUNG

1. Tổng quan về bộ nhớ ảo
2. Cài đặt bộ nhớ ảo: Demand Paging
3. Các giải thuật thay trang (Page Replacement Algorithms)
4. Vấn đề cấp phát Frames
5. Vấn đề Thrashing

Tổng quan về bộ nhớ ảo

1

Dynamic loading

- Cơ chế: chỉ khi nào cần được gọi đến thì một thủ tục mới được nạp vào bộ nhớ chính ⇒ tăng độ hiệu dụng của bộ nhớ bởi vì các thủ tục không được gọi đến sẽ không chiếm chỗ trong bộ nhớ
- Rất hiệu quả trong trường hợp tồn tại khối lượng lớn mã chương trình có tần suất sử dụng thấp, không được sử dụng thường xuyên (ví dụ các thủ tục xử lý lỗi)
- Hỗ trợ từ hệ điều hành
  - Thông thường, user chịu trách nhiệm thiết kế và hiện thực các chương trình có dynamic loading.
  - Hệ điều hành chủ yếu cung cấp một số thủ tục thư viện hỗ trợ, tạo điều kiện dễ dàng hơn cho lập trình viên.

Tổng quan về bộ nhớ ảo

- Nhận xét: không phải tất cả các phần của một process cần thiết phải được nạp vào bộ nhớ chính tại cùng một thời điểm
- Ví dụ:
  - Đoạn mã điều khiển các lỗi hiếm khi xảy ra
  - Các arrays, list, tables được cấp phát bộ nhớ (cấp phát tĩnh) nhiều hơn yêu cầu thực sự
  - Một số tính năng ít khi được dùng của một chương trình
  - Cả chương trình thì cũng có đoạn code chưa cần dùng
- Bộ nhớ ảo (virtual memory): Bộ nhớ ảo là một kỹ thuật cho phép xử lý một tiến trình không được nạp toàn bộ vào bộ nhớ vật lý

Tổng quan về bộ nhớ ảo (tt)

- Ưu điểm của bộ nhớ ảo
  - Số lượng process trong bộ nhớ nhiều hơn
  - Một process có thể thực thi ngay cả khi kích thước của nó lớn hơn bộ nhớ thực
  - Giảm nhẹ công việc của lập trình viên
- Không gian tráo đổi giữa bộ nhớ chính và bộ nhớ phụ (swap space)
- Ví dụ:
  - swap partition trong Linux
  - file pagefile.sys trong Windows

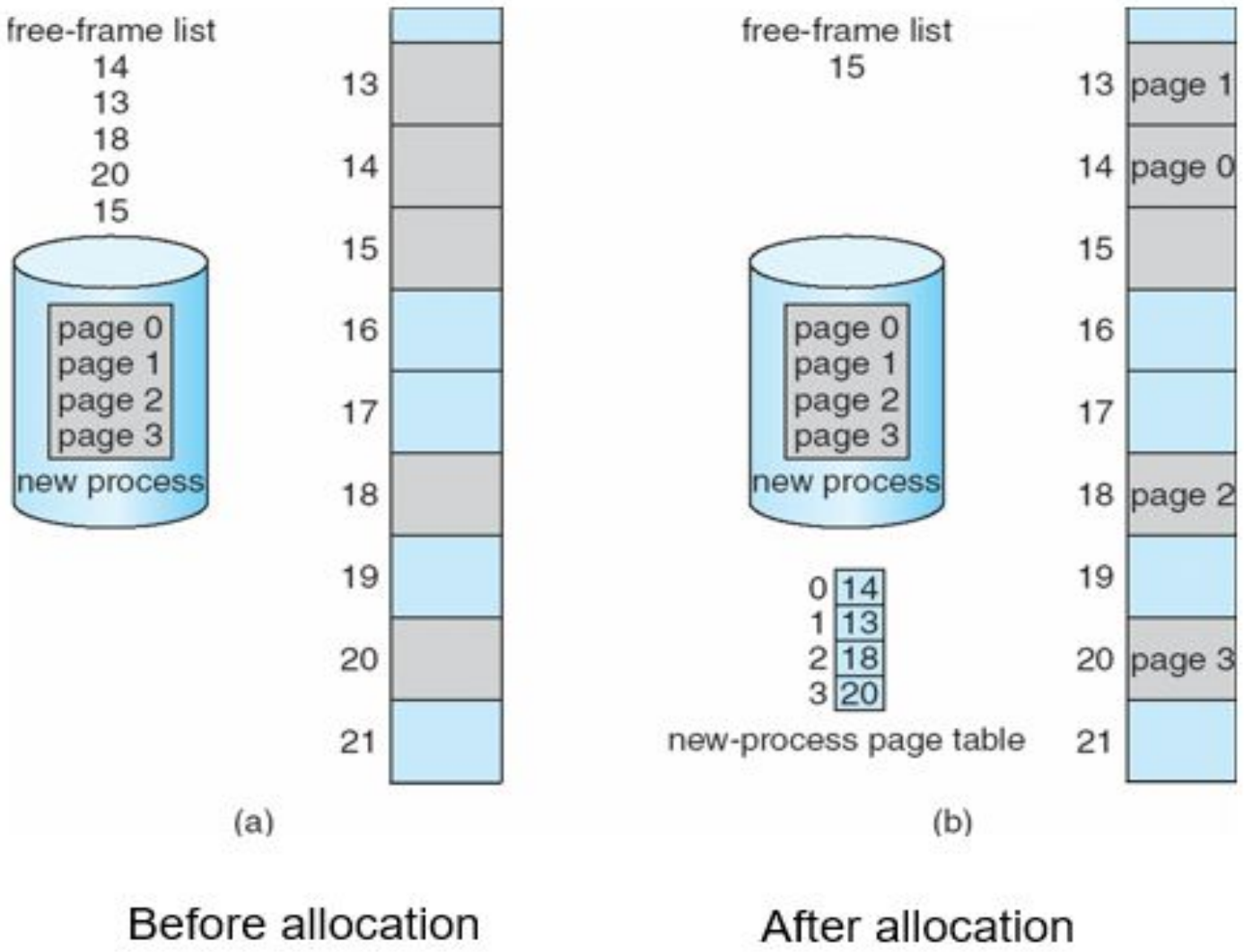
Cài đặt bộ nhớ ảo

2

Cài đặt bộ nhớ ảo

- Có hai kỹ thuật:
  - Phân trang theo yêu cầu (Demand Paging)
  - Phân đoạn theo yêu cầu (Demand Segmentation)
- Phần cứng memory management phải hỗ trợ paging và/hoặc segmentation
- OS phải quản lý sự di chuyển của trang/đoạn giữa bộ nhớ chính và bộ nhớ thứ cấp
- Trong chương này,
  - Chỉ quan tâm đến paging
  - Phần cứng hỗ trợ hiện thực bộ nhớ ảo
  - Các giải thuật của hệ điều hành

Cơ chế phân trang



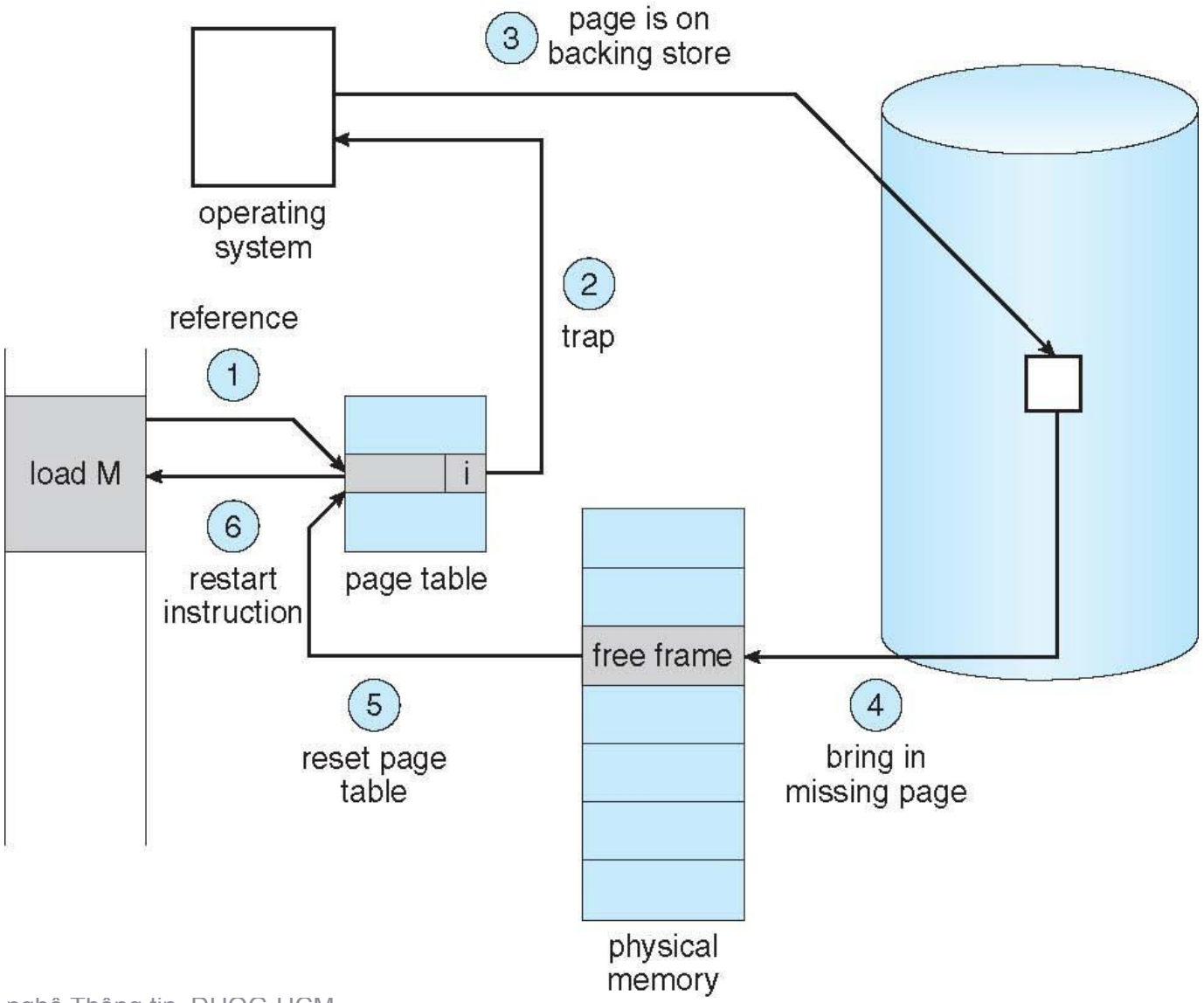
Before allocation

After allocation

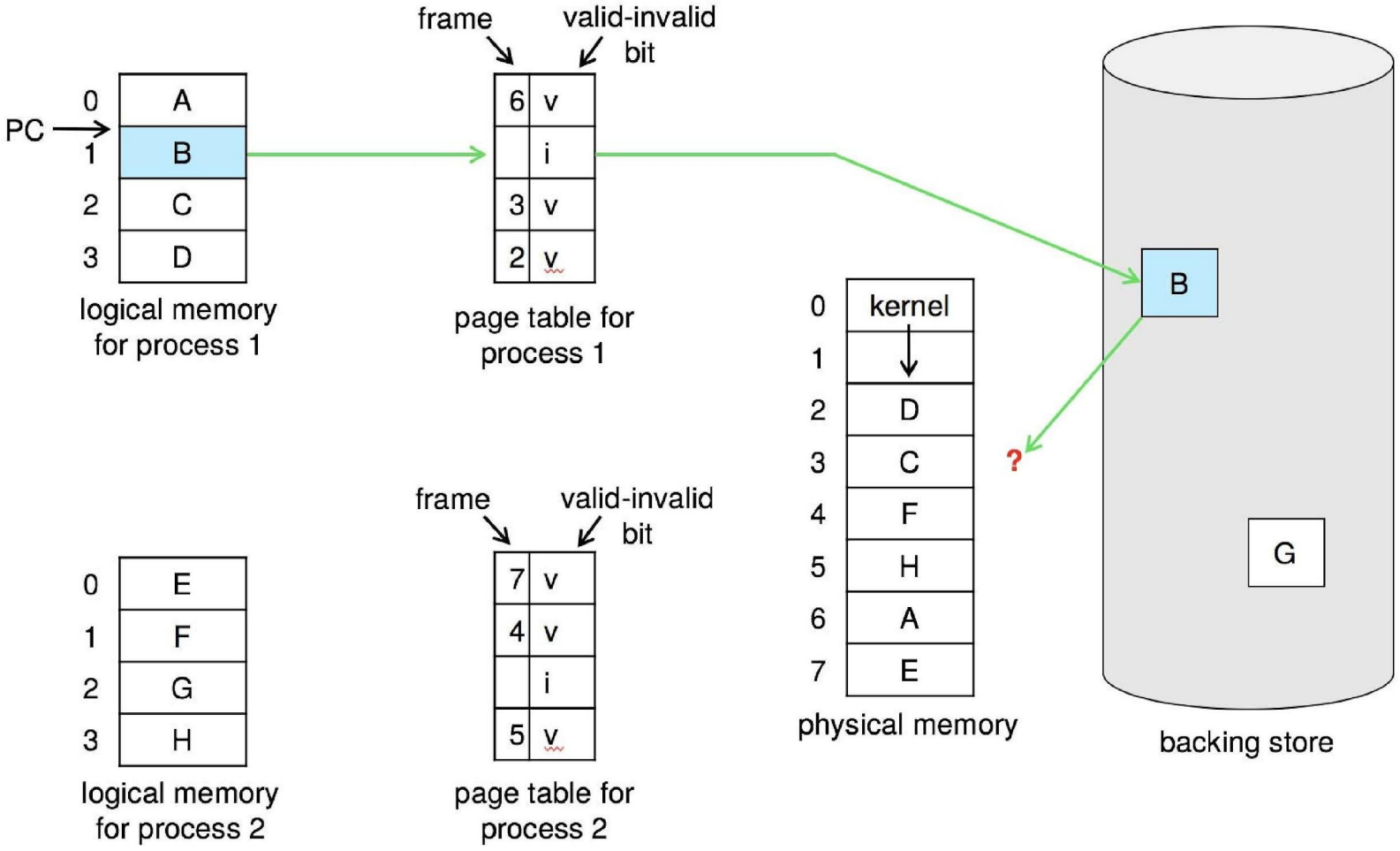
Phân trang theo yêu cầu

- Demand paging: các trang của tiến trình chỉ được nạp vào bộ nhớ chính khi được yêu cầu.
- Khi có một tham chiếu đến một trang mà không có trong bộ nhớ chính (valid bit) thì phần cứng sẽ gây ra một ngắt (gọi là page-fault trap) kích khởi page-fault service routine (PFSR) của hệ điều hành.
- PFSR:
  - Bước 1: Chuyển process về trạng thái blocked
  - Bước 2: Phát ra một yêu cầu đọc đĩa để nạp trang được tham chiếu vào một frame trống; trong khi đợi I/O, một process khác được cấp CPU để thực thi
  - Bước 3: Sau khi I/O hoàn tất, đĩa gây ra một ngắt đến hệ điều hành; PFSR cập nhật page table và chuyển process về trạng thái ready.

Lỗi trang và các bước xử lý



Khi cần thay thế trang

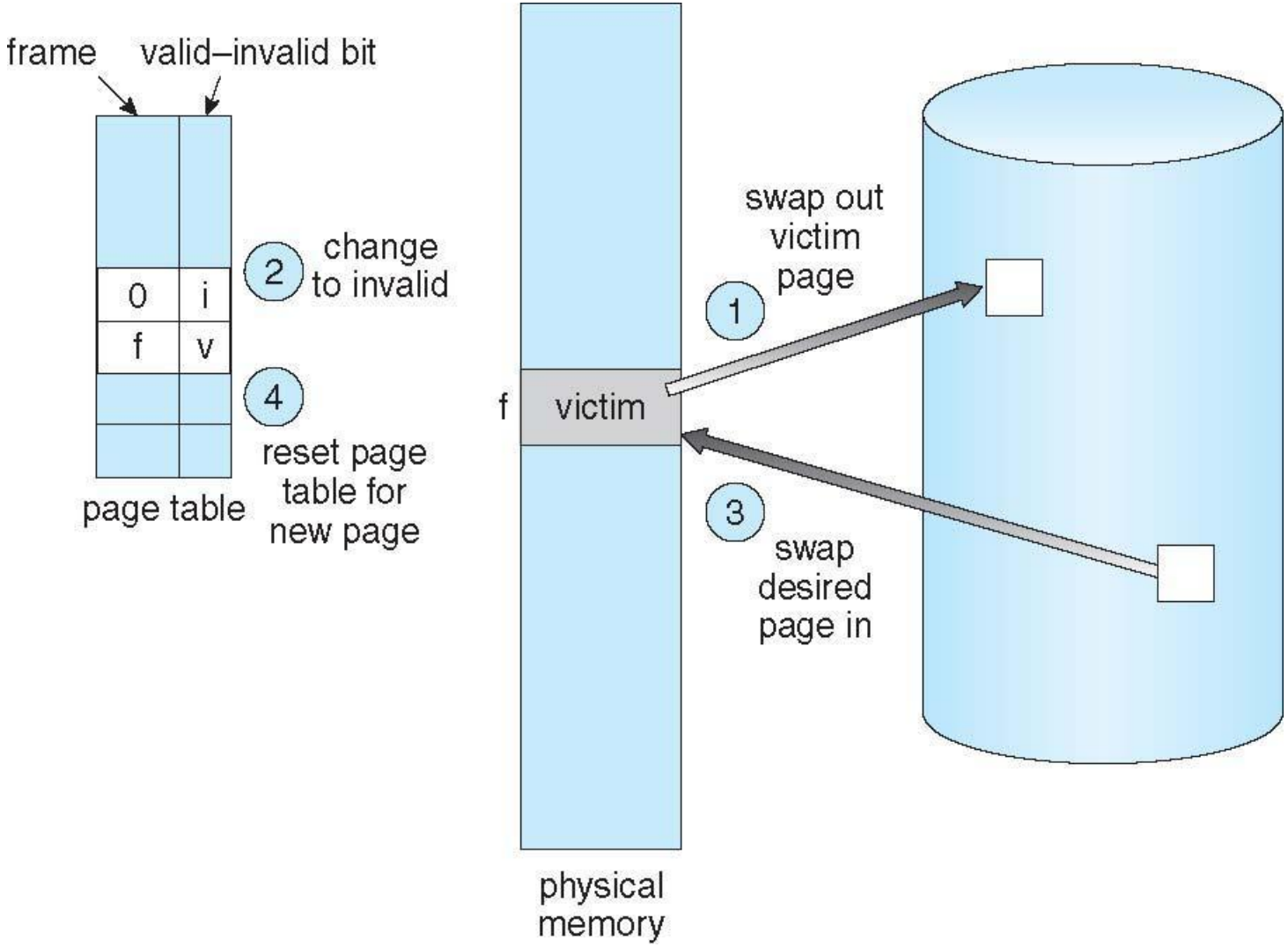


Thay thế trang nhớ

Bước 2 của PFSR giả sử phải thay trang vì không tìm được frame trống, PFSR được bổ sung như sau:

- Xác định vị trí trên đĩa của trang đang cần
- Tìm một frame trống:
  - Nếu có frame trống thì dùng nó
  - Nếu không có frame trống thì dùng một giải thuật thay trang để chọn một trang hy sinh (victim page)
  - Ghi victim page lên đĩa; cập nhật page table và frame table tương ứng
- Đọc trang đang cần vào frame trống (đã có được từ bước 2); cập nhật page table và frame table tương ứng.

Thay thế trang nhớ (tt)



Thay thế trang nhớ (tt)

Hai vấn đề chủ yếu:

- Frame-allocation algorithm
  - Cấp phát cho process bao nhiêu frame của bộ nhớ thực?
- Page-replacement algorithm
  - Chọn frame của process sẽ được thay thế trang nhớ
  - Mục tiêu: số lượng page-fault nhỏ nhất
  - Được đánh giá bằng cách thực thi giải thuật đối với một chuỗi tham chiếu bộ nhớ (memory reference string) và xác định số lần xảy ra page fault

Thay thế trang nhớ (tt)

Ví du

Thứ tự tham chiếu các địa chỉ nhớ, với page size = 100:

0098, 0432, 0201, 0612, 0302, 0103, 0104, 0101, 0611, 0102, 0103, 0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105

các trang nhớ sau được tham chiếu lần lượt = chuỗi tham chiếu bộ nhớ (trang nhớ)

0, 4, 2, 6, 3,

1, 1, 1, 6, 1,

1, 1, 1, 6, 1,

1, 1, 1, 6, 1,

1

Các giải thuật thay trang

3

Các giải thuật thay trang

- Giải thuật thay trang FIFO
- Giải thuật thay trang OPT
- Giải thuật thay trang LRU
- Các dữ liệu cần biết ban đầu:
  - Số khung trang
  - Tình trạng ban đầu
  - Chuỗi tham chiếu

Giải thuật thay trang FIFO

- Xét một tiến trình có 8 trang và chuỗi tham chiếu bộ nhớ cần truy xuất như sau:

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

- Số khung trang là 3 frames
- Ban đầu các khung trang đều trống

Giải thuật thay trang FIFO

Giải thuật thay trang FIFO sẽ thay thế trang nhớ có thời gian vào sớm nhất trong các trang nhớ trong 3 khung trang

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
	0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
		1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
*	*	*	*		*	*	*	*	*	*			*	*			*	*	*

Nghịch lý Belady

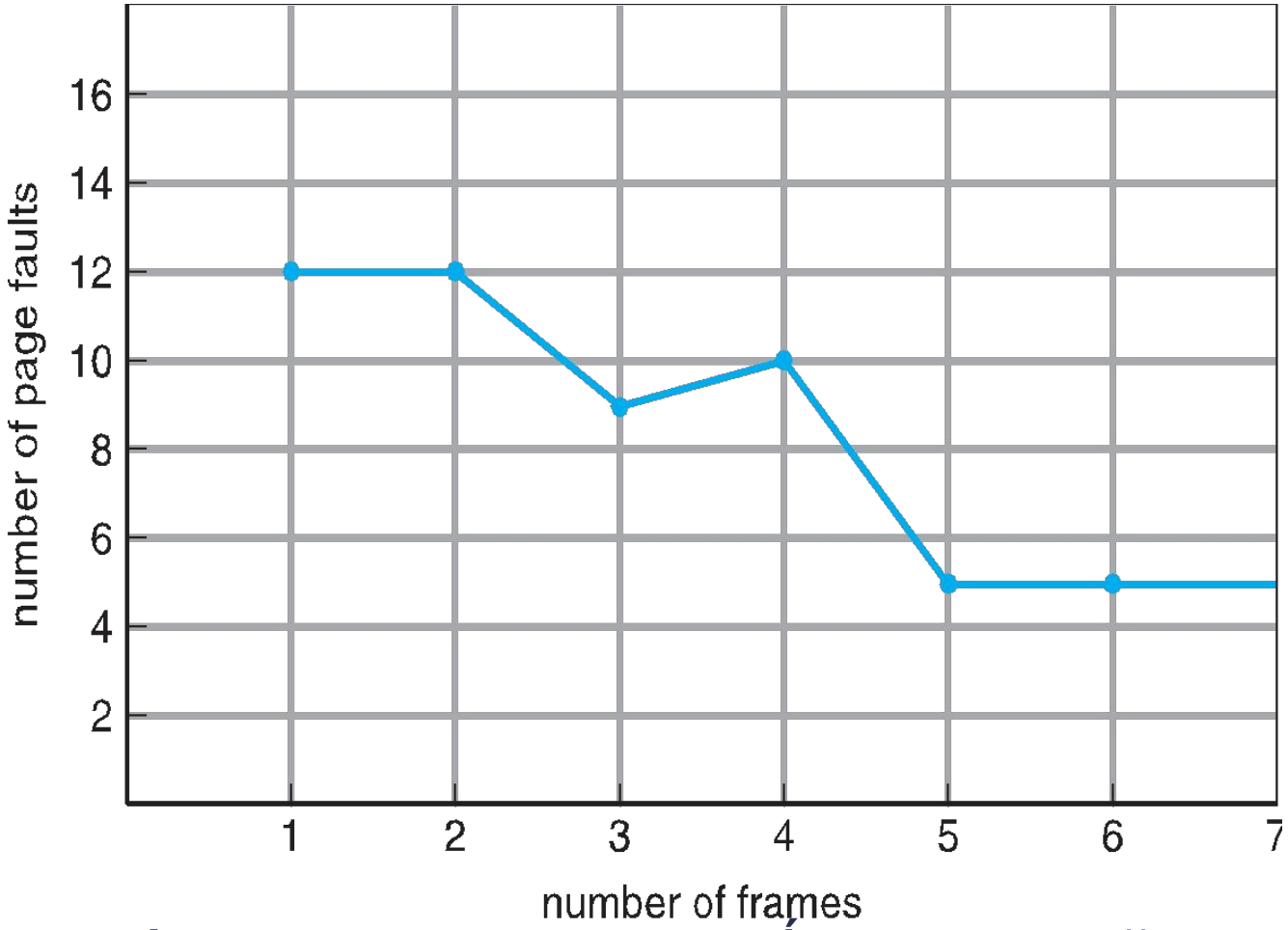
Sử dụng 3 khung trang , sẽ có 9 lỗi trang phát sinh

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4
*	*	*	*	*	*	*			*	*	

Sử dụng 4 khung trang , sẽ có 10 lỗi trang phát sinh

1	2	3	4	1	2	5	1	2	3	4	5
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3
*	*	*	*			*	*	*	*	*	*

Nghịch lý Belady (tt)



Bất thường (anomaly) Belady: số page fault tăng mặc dầu quá trình đã được cấp nhiều frame hơn.

Giải thuật thay trang OPT

Giải thuật thay trang OPT sẽ thay thế trang nhớ sẽ được tham chiếu trễ nhất trong tương lai -> cần phải biết trước các trang sẽ được tham chiếu trong tương lai

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
	0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
		1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
*	*	*	*		*		*			*			*				*		

Giải thuật thay trang LRU

- Mỗi trang được ghi nhận (trong bảng phân trang) thời điểm được tham chiếu ⇒ trang LRU là trang nhớ có thời điểm tham chiếu nhỏ nhất (OS tốn chi phí tìm kiếm trang nhớ LRU này mỗi khi có page fault)
- Do vậy, LRU cần sự hỗ trợ của phần cứng và chi phí cho việc tìm kiếm LRU

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
		1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
*	*	*	*		*		*	*	*	*			*		*		*		

LRU và FIFO

So sánh các giải thuật thay trang LRU và FIFO

Page address stream	2	3	2	1	5	2	4	5	3	2	5	2
LRU	2	2	2	2	2	2	2	2	3	3	3	3
		3	3	3	5	5	5	5	5	5	5	5
				1	1	1	4	4	4	2	2	2
					F		F		F	F		
FIFO	2	2	2	2	5	5	5	5	3	3	3	3
		3	3	3	3	2	2	2	2	2	5	5
				1	1	1	4	4	4	4	4	2
					F	F	F		F		F	F

Vấn đề cấp phát Frames

4

Số lượng frame cấp cho process

- OS phải quyết định cấp cho mỗi process bao nhiêu frame.
  - Cấp ít frame ⇒ nhiều page fault
  - Cấp nhiều frame ⇒ giảm mức độ multiprogramming
- Chiến lược cấp phát tĩnh (fixed-allocation)
  - Số frame cấp cho mỗi process không đổi, được xác định vào thời điểm loading và có thể tùy thuộc vào từng ứng dụng (kích thước của nó,…)
- Chiến lược cấp phát động (variable-allocation)
  - Số frame cấp cho mỗi process có thể thay đổi trong khi nó chạy
    - Nếu tỷ lệ page-fault cao ⇒ cấp thêm frame
    - Nếu tỷ lệ page-fault thấp ⇒ giảm bớt frame
  - OS phải mất chi phí để ước định các process

Chiến lược cấp phát tĩnh

- Cấp phát bằng nhau: Ví dụ, có 100 frame và 5 process → mỗi process được 20 frame
- Cấp phát theo tỉ lệ: dựa vào kích thước process

$$s_i = \text{size of process } p_i$$

$$S = \sum s_i$$

$$m = \text{total number of frames}$$

$$a_i = \text{allocation for } p_i = \frac{s_i}{S} \times m$$

Ví dụ:

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 64 \approx 5$$

$$a_2 = \frac{127}{137} \times 64 \approx 59$$

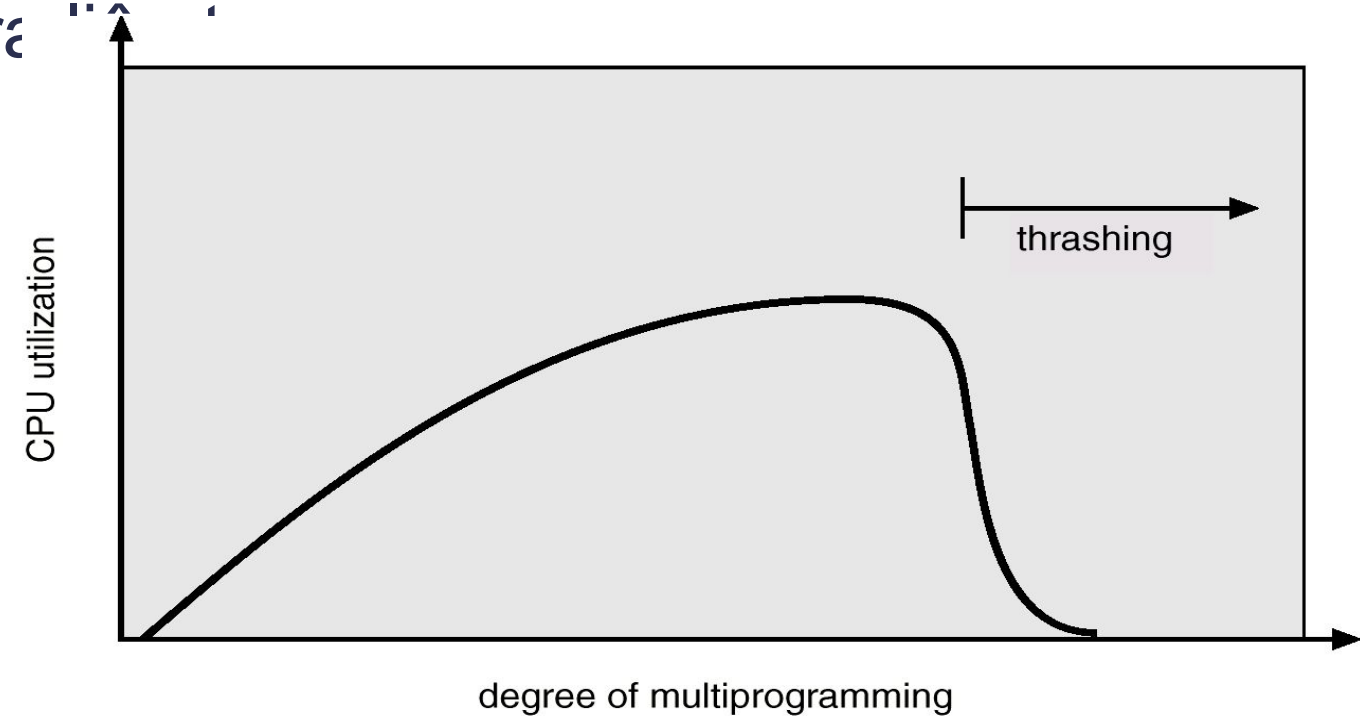
- Cấp phát theo độ ưu tiên

Vấn đề Thrashing

5

Trì trệ trên toàn bộ hệ thống

- Nếu một process không có đủ số frame cần thiết thì tỉ số page faults/sec rất cao.
- Thrashing: hiện tượng các trang nhớ của một process bị hoán chuyển vào/ra liên tục



Mô hình cục bộ

- Để hạn chế thrashing, hệ điều hành phải cung cấp cho process càng "đủ" frame càng tốt. Bao nhiêu frame thì đủ cho một process thực thi hiệu quả?
- Nguyên lý locality (locality principle)
  - Locality là tập các trang được tham chiếu gần nhau
  - Một process gồm nhiều locality, và trong quá trình thực thi, process sẽ chuyển từ locality này sang locality khác
- Vì sao hiện tượng thrashing xuất hiện? Khi Σ size of locality > memory size

Giải pháp tập làm việc

- Được thiết kế dựa trên nguyên lý locality.
- Xác định xem process thực sự sử dụng bao nhiêu frame.
- Định nghĩa:
  - WS(t) - các tham chiếu trang nhớ của process gần đây nhất cần được quan sát.
  - Δ - khoảng thời gian tham chiếu
- Ví dụ:

$\square = 4$

chuỗi tham khảo trang nhớ

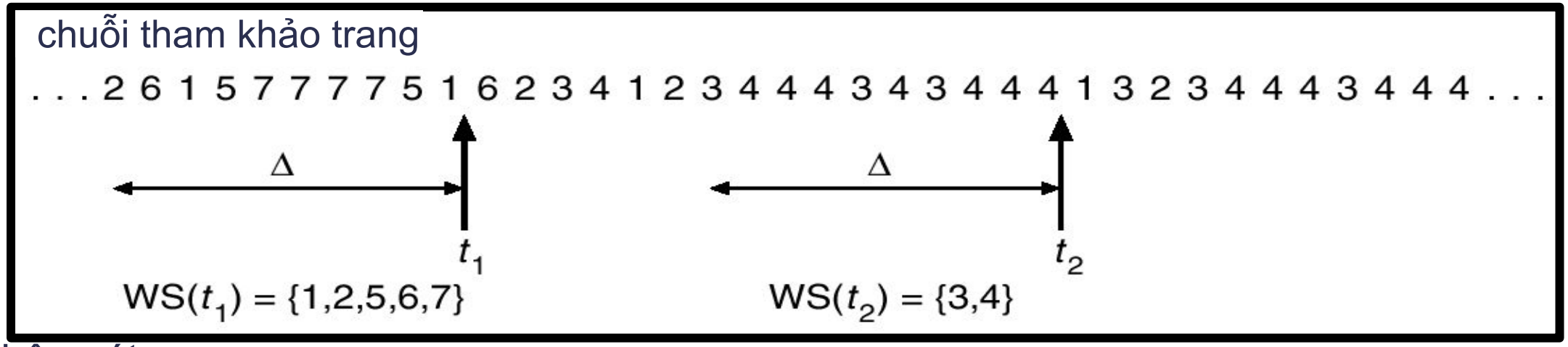
2 4 $\boxed{5\ 6\ 9\ 1}$ 3 2 6 3 9 2 1 4

thời điểm t_1

Giải pháp tập làm việc (tt)

- Định nghĩa: Working set của process Pi , ký hiệu WSi, là tập gồm Δ các trang được sử dụng gần đây nhất.

  Ví dụ: Δ = 10 và

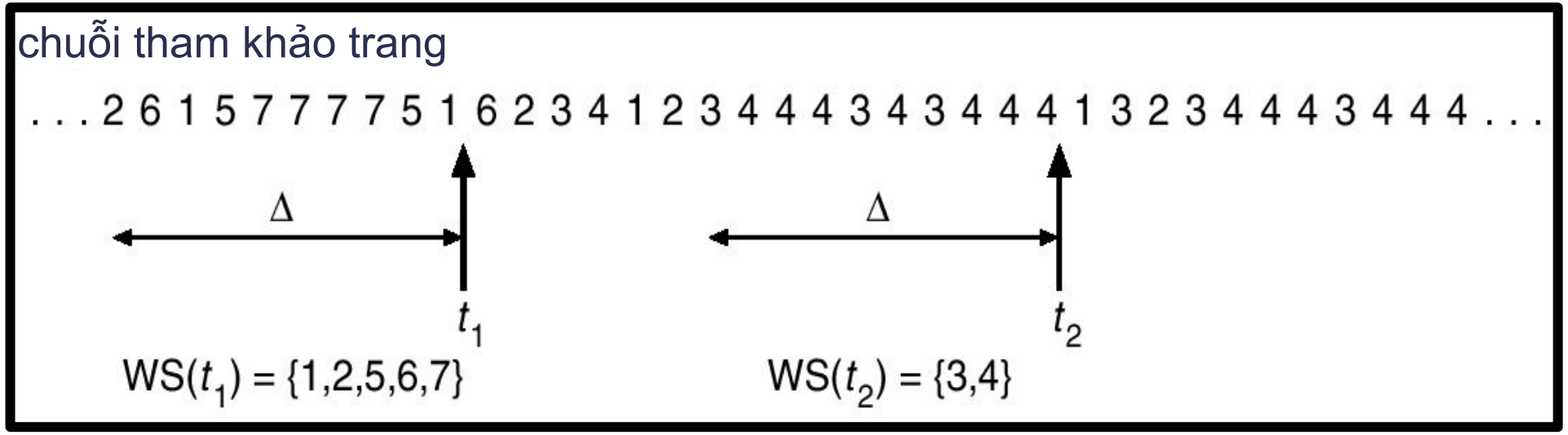


- Nhận xét:
  - Δ quá nhỏ ⇒ không đủ bao phủ toàn bộ locality.
  - Δ quá lớn ⇒ bao phủ nhiều locality khác nhau.
  - Δ = ∞ ⇒ bao gồm tất cả các trang được sử dụng.
- Dùng working set của một process để xấp xỉ locality của nó

Giải pháp tập làm việc (tt)

- Định nghĩa: WSSi là kích thước của working set của Pi:
  - WSSi = số lượng các trang trong WSi

Ví dụ: Δ = 10 và



$WSS(t_1) = 5$ $WSS(t_2) = 2$

Giải pháp tập làm việc (tt)

- Đặt D = Σ WSSi = tổng các working-set size của mọi process trong hệ thống.
  - Nhận xét: Nếu D > m (số frame của hệ thống) ⇒ sẽ xảy ra thrashing.
- Giải pháp working set:
  - Khi khởi tạo một quá trình: cung cấp cho quá trình số lượng frame thỏa mản working-set size của nó.
  - Nếu D > m ⇒ tạm dừng một trong các process.
    - Các trang của quá trình được chuyển ra đĩa cứng và các frame của nó được thu hồi.

Giải pháp tập làm việc (tt)

- WS loại trừ được tình trạng trì trệ mà vẫn đảm bảo mức độ đa chương
- Theo vết các WS? => WS xấp xỉ (đọc thêm trong sách)
- Đọc thêm:
  - Hệ thống tập tin
  - Hệ thống nhập xuất
  - Hệ thống phân tán

Tóm tắt lại nội dung buổi học

- Tổng quan về bộ nhớ ảo
- Cài đặt bộ nhớ ảo: Demand Paging
- Các giải thuật thay trang (Page Replacement Algorithms)
- Vấn đề cấp phát Frames
- Vấn đề Thrashing

Bài tập 1

Xét chuỗi truy xuất bộ nhớ sau:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1

Có bao nhiêu lỗi trang xảy ra khi sử dụng các thuật toán thay thế sau đây, giả sử có lần lượt là 4 khung trang.

a. LRU

b. FIFO

c. Chiến lược tối ưu (OPT)

THẢO LUẬN